Country Service – Onboarding Task

Overview

Build a REST API using Quarkus that serves country information. The service fetches data from an external REST API (restcountries.com) on startup, persists it in a PostgreSQL database, and exposes it through REST endpoints. Additionally, it integrates with a SOAP web service to retrieve country data by ISO code.

Functional Requirements

1. Startup Data Loader

On application start, call the external REST API at:
`https://restcountries.com/v3.1/all?fields=name,currencies`
and persist the results into the database. Skip if data already exists.

2. REST Client (MicroProfile)

Create an interface-based REST client that calls restcountries.com for (1).

3. REST Endpoints

Expose the following under `/api/countries`:

Method	Path	Description
GET	`/api/countries?page=0&size=10`	Paginated list of all countries
GET	`/api/countries/{country}`	Find a country by name using fuzzy matching
GET	`/api/countries/currency/{currencyCode}`	Countries using a given currency
GET	`/api/countries/soap/code/{code}`	Country name via SOAP by ISO code
GET	`/api/countries/soap/code/`	All countries via SOAP

4. SOAP Client

Use the provided WSDL (src/main/resources/wsdl/CountryInfoService.wsdl) to generate Java classes at build time. Create a wrapper client class that uses @CXFClient to call the SOAP service.

The SOAP service WSDL is available at:
http://webservices.oorsprong.org/websamples.countryinfo/CountryInfoService.wso

Your wrapper should expose at least:

- A method that returns a country's name given its ISO code (e.g. "GR" → "Greece")
- A method that returns all countries with their ISO codes

These are then consumed by the REST endpoints under /api/countries/soap/.

5. MapStruct Mapper

Use MapStruct for all your mappings, these can be converting DTOs (object to be received in your controller) to Entities in the database and the reverse.

6. Exception Handling

Create a global exception mapper (@Provider) using @ServerExceptionMapper for:

Exception	**HTTP Status**
`CountryNotFoundException`	404
`CountryLoadException`	500
`Generic Throwable fallback`	500

Return a structured JSON error response with message, statusCode, and timestamp.

7. Tests

Provide both unit and integration tests for your project.

Unit tests:

- Test your MapStruct mapper logic (e.g. does a CountryDTO correctly map to a Country entity and vice versa?)

Integration tests:

- Write tests using @QuarkusTest + REST Assured that hit your actual endpoints.
- Use WireMock to stub the external REST client so tests don't depend on the real API.
- Verify correct HTTP status codes and response bodies for both success and error cases

8. Running locally

Test your API locally using Insomnia.

Stack

1) Quarkus 3.x (Java 17)
2) PostgreSQL (you can use quarkus dev services for Postgres (look that up) or else you can spin up your own Postgres via Podman or docker (look that up as well, you may use AI)
3) Hibernate ORM with Panache
4) MapStruct for DTO/Entity mapping
5) Quarkus CXF for SOAP client
6) MicroProfile REST Client for external REST calls
7) WireMock for testing

Suggested Project Structure

Pick any microservice in the VF-Technology-Hub and look at the project folder structure. Also, try and follow the same code conventions (naming, use of Optional etc)

Some prework (in no particular order)

1) Testing: What is unit vs integration testing, why use WireMock in Integration tests
    a. Videos:
    https://confluence.sp.vodafone.com/spaces/VGDALPD/pages/207281162/Knowledge+Vault
    Under section “Testing basics”
2) Quarkus basics:
    a. Resource, service, entity, repository layers, how they differ
    b. Basic dependency injection (@ApplicationScoped, @Injection) and how they work.
3) Persistence basics:
    a. What an entity is
    b. Basic JPA relationships
    c. What is a repository layer
    d. How panache simplifies operations
4) DTO/entity mapping:
    a. Why DTOs are used instead of exposing database entities directly
    b. Basic MapStruct Usage
5) REST, SOAP APIs and SOAP Apis in practice with Quarkus CXF.
6) Writing a REST Client with Quarkus.
7) Error handling: what global exception handling is and why structured error responses are useful
8) Databases:

    a. Differences between quarkus dev and running Postgres manually

Bonus section (optional)

1) Use Git and GitHub throughout the project. Create a repository, work with feature branches (e.g. feature/rest-endpoints, feature/soap-client), and open pull requests to merge into main.
2) Use versioning in your API.
3) Run locally your own Postgres using Docker or Podman and connect it to your application.
4) Understand Quarkus under-the-hood on how threads are utilized.
5) Use Redis to cache frequently used countries.
6) Study and apply reactive coding to the project with Mutiny: https://quarkus.io/guides/mutiny-primer
    a. If you decide to pursue this, provide two projects, one with the synchronous approach and one with the asynchronous approach.

References

1. Getting Started: https://quarkus.io/guides/getting-started
2. Quarkus REST: https://quarkus.io/guides/rest
3. REST Client: https://quarkus.io/guides/rest-client
4. Hibernate ORM with Panache: https://quarkus.io/guides/hibernate-orm-panache
5. Quarkus CXF: https://docs.quarkiverse.io/quarkus-cxf/dev/index.html
6. MapStruct Reference Guide: https://mapstruct.org/documentation/stable/reference/html/
7. OpenAPI and Swagger UI: https://quarkus.io/guides/openapi-swaggerui
8. Dev Services for Databases: https://quarkus.io/guides/databases-dev-services
9. WireMock Getting Started: https://wiremock.org/docs/getting-started/
10. Testing Your Application: https://quarkus.io/guides/getting-started-testing